

Low-Latency Low-Randomness OPINI Gadgets and Their Formal Verification

Lixuan Wu^{1,2,3}, Yanhong Fan^{1,2,3,4}, Guowei Liu^{1,2,3},
Chaoran Wang^{1,2,3} and Meiqin Wang^{1,2,3}

¹ School of Cyber Science and Technology, Shandong University, Qingdao, China,
lixuanwu@sdu.edu.cn, yanhongfan@sdu.edu.cn, guoweiliu@mail.sdu.edu.cn

² State Key Laboratory of Cryptography and Digital Economy Security,
Shandong University, Qingdao, 266237, China,

³ Key Laboratory of Cryptologic Technology and Information Security,
Ministry of Education, Shandong University, Jinan, China,

⁴ Quan Cheng Shandong Laboratory, Jinan, China

Abstract. Masking is an essential countermeasure against side-channel attacks, yet implementing secure and low-latency hardware masking remains challenging. In particular, although OPINI provides strong composability guarantees for single-cycle iterative architectures, prior low-latency OPINI gadget, HPC4, is limited to two-input multiplication. In this work, we present a low-latency, low-randomness, first-order OPINI gadget applicable to arbitrary Boolean functions, denoted as GOM. Independent and concurrent work by Rahimi and Moradi proposes OTSM, which is also a generic, low-latency first-order OPINI gadget. Our construction involves two new techniques: (i) extending the HPC4 idea—originally masking each share of one secret input with two bits of randomness—to masking each shared monomial derived from the input shares accordingly, and (ii) a randomness-reassignment technique that enables the two circuits generating the output shares to reuse the same set of randomness while preserving OPINI security. To validate OPINI security, we propose a formal verification technique based on three symbolic reduction rules, and use it to verify multiple low-latency OPINI gadgets (i.e., HPC4, GOM and OTSM). Leveraging the generality of our gadget, we instantiate several OPINI-secure S-boxes across different algebraic degrees. For the algebraic-degree-2 Ascon S-box, our gadget achieves a 21% reduction in area and a 28% reduction in randomness compared to the HPC4-based implementation. We further construct higher-degree S-boxes from the PRESENT, PRINCE and AES ciphers and report their hardware performance as reference baselines. We also provide an apples-to-apples comparison with OTSM. All masked S-boxes are successfully verified within 20 minutes using our formal verification method. Finally, FPGA-based experiments confirm the practical security of the masked implementations.

Keywords: Hardware · Low-Latency · OPINI Gadgets · Formal Verification

1 Introduction

Masking [CJRR99, GP99], rooted in secret sharing [Sha79], is one of the most popular countermeasures against Side-Channel Analysis (SCA) attacks [Koc96, KJJ99]. In a masking scheme, computations operate on multiple randomized shares of sensitive data rather than on the data itself, thereby removing exploitable statistical dependencies between intermediate values and side-channel information. Threshold Implementation (TI) [NRR06] is the first masking strategy to avoid leakage in hardware implementations. Subsequent schemes, including Consolidating Masking Schemes (CMS) [RBN⁺15] and

Domain-Oriented Masking (DOM) [GMK16], further reduced hardware overhead without compromising security.

Despite these advances in masking strategies, constructing and proving secure masking for arbitrary Boolean functions remains challenging due to the prohibitive computational complexity involved. To address this, Cassiers et al. [CS20] introduced Probe-Isolating Non-Interference (PINI), an elegant notion that enables a divide-and-conquer approach for constructing large-scale masked circuits. Small PINI gadgets (e.g., HPC1 [CGLS21], HPC2 [CGLS21] and HPC3 [KM22]) can be modularly composed into larger secure circuits. Since these gadgets primarily focus on two-input multiplication, circuits with high algebraic degree typically require multi-cycle latency (e.g., a 6-cycle AES S-box constructed from HPC2 gadgets [MCS22]).

To mitigate such latency overhead, Knichel et al. [KSM22] introduced the Low-Latency Generic Private Circuits (GHPC_{LL}), the first PINI-secure construction for arbitrary Boolean functions using only a single layer of registers. However, masking schemes based on GHPC_{LL} have a large implementation area and many random bits. More recently, Time Sharing Masking (TSM) [VDB⁺24] has further reduced hardware area and fresh randomness costs while maintaining single-cycle latency. The low-latency feature has also been extended to higher-order gadgets with minimal additional latency [VDB⁺25, LMDM25]

However, as emphasized in [CS21], satisfying PINI may not be sufficient to guarantee security in single-cycle iterative architectures, where transitions introduce additional leakage. This limitation motivated the authors to introduce a stronger composability notion, Output-Probe-Isolating Non-Interference (OPINI), which enables trivial and secure composition even in a single-cycle-per-round implementation. A representative low-latency OPINI gadget is HPC4 [CSV24], which supports two-input multiplication across arbitrary security order. Independent and concurrent with our work, Rahimi and Moradi [RM26] propose OTSM, a low-latency first-order OPINI gadget that can be extended to arbitrary Boolean functions.

1.1 Contributions

The paper proposes a construction method for the low-latency, first-order OPINI gadget applicable to arbitrary Boolean functions. The proposed Generic OPINI Masking (GOM)¹ gadget extends the capability of OPINI-secure constructions beyond the two-input multiplication while maintaining practical efficiency in terms of latency, randomness, and area. In addition, we propose a symbolic formal verification technique to validate the OPINI security of GOM. Our contributions are summarized below.

A Generic OPINI Gadget for Arbitrary Boolean Functions. Our gadget construction extends the HPC4 idea by masking each shared monomial derived from input shares with two random bits to achieve the OPINI security under the single-cycle latency. We further introduce a randomness-reassignment technique that enables the two circuits producing the output shares to reuse the same randomness, thereby reducing the total randomness without compromising security. Based on these techniques, we construct a generic first-order OPINI gadget that achieves both low-latency and low-randomness.

A Symbolic Formal Verification for the GOM Gadget. To validate the OPINI security, we propose a symbolic verification technique. The technique introduces three symbolic reduction rules to derive the information required to simulate each probe configuration in a target gadget. If the resulting information satisfies the OPINI definition, the gadget is considered OPINI-secure. Our technique applies to single-cycle OPINI gadgets, including HPC4, GOM, OTSM.

Improved Hardware Performance. Leveraging the generality of GOM, we instantiate several masked S-boxes across different algebraic degrees. For the algebraic-degree-2

¹Since our construction method serves as an OPINI-secure gadget for generic Boolean functions, we refer to it as Generic OPINI Masking (GOM).

Ascon S-box, our gadget achieves a 21% area reduction and 28% randomness reduction compared to the HPC4-based implementation. We further construct higher-algebraic-degree (> 2) masked S-boxes, including the 4-bit PRINCE S-box and the 8-bit AES S-box, report their hardware performance as reference baselines, and provide a comparison with the concurrent work OTSM. All instantiated S-boxes are successfully verified within 20 minutes using our symbolic verification technique.

1.2 Outline

Section 2 introduces the necessary notations and background for understanding this work. In Section 3, we present the low-latency OPINI gadget for arbitrary Boolean functions, and the symbolic formal verification technique used to validate its security. Section 4 instantiates several masked S-boxes using our gadget, reports their hardware performance, and verifies their security using our formal technique. In Section 5, we conduct FPGA-based experiments to confirm the practical security of our gadgets.

2 Preliminaries

2.1 Notations

We denote a Boolean variable by a lower case letter $x \in \mathbb{F}_2$ and a vector of Boolean variables by an uppercase letter $X \in \mathbb{F}_2^n$. The i -th share of the unshared variable X is written using a subscript, i.e., X_i . The superscript notation X_j denotes the j -th element of the vector X . The set of all shares of all elements of vector X is denoted as $Sh(X)$. In Boolean masking, a Boolean variable x is split into $d + 1$ uniform random shares, denoted as $(x_0, x_1, \dots, x_d)$. To ensure correctness, these shares satisfy the condition $x = \bigoplus_{i=0}^d x_i$.

2.2 Probing Model

Several adversary models have been introduced to evaluate the security of masking schemes and to guide the construction of more masking schemes. Among them, the d -probing model is widely used for analyzing and designing masking schemes in software implementations. In this model, the parameter d , representing the number of probes, determines the order of the attack. To capture information leakage caused by physical defaults such as glitches, transitions, and couplings, Faust et al. [FGP⁺18] introduced the robust probing model. In this model, each physical default is abstracted using a probe extension, which increases the number of values observable through a single probe.

In this work, we focus on the glitches and transitions. Glitches refer to the unpredictable signal transition activities on combinational wires, caused by the different signal delays of logic gates. Transitions are the unpredictable recombinations of register contents across consecutive invocations (e.g., clock cycles). A glitch-extended probe placed on a combinational wire observes all intermediate values up to the most recent synchronization point, such as registers and primary inputs. A transition-extended probe placed on a register observes a pair of values that correspond to consecutive invocations.

2.3 Trivially Composable Masking

Constructing masked circuits with large functions or high security orders remains challenging, due to the prohibitive computational complexity involved. For this, researchers have introduced some composability notions that enable the construction of large circuits by combining smaller sub-circuits (often referred to as gadgets) with particular properties. To support trivial composability, Barthe et al. [BBD⁺16] proposed the notion of Strong Non-Interference (SNI), which ensures secure composability among single-output gadgets.

This concept was later extended to multiple-output gadgets by Cassiers et al. [CS20]. In the same work, the authors also introduced a new composable notion, Probe-Isolating Non-Interference (PINI), which further reduces the implementation overhead of gadgets while preserving trivial composability. As elaborated in [CS21], satisfying PINI may not be sufficient to guarantee security in an iterative architecture, where one input of a gadget depends on an output of itself, introducing leakage from transitions. To solve this issue, the authors proposed OPINI, a stronger composability notion that accounts for both glitches and transitions and guarantees security in such scenarios. As a necessary concept, we first recall the concept of simulability before introducing PINI and OPINI.

Definition 1 (Simulability [CS20]). Let $P = \{p_0, p_1, \dots, p_{l-1}\}$ be a set of l probes placed on a gadget G , and let G_P denote the tuple of probe values obtained during an execution of G . We say that P is simulatable by a set of input shares S if there exists a randomized simulator $SIM : \mathbb{F}_2^{|S|} \rightarrow \mathbb{F}_2^l$ such that, for any shared input of G , the distributions of G_P and $SIM(S)$ are identical, where the probability is taken over the random bits in G and SIM .

Definition 2 (d -Probe-Isolating Non-Interference [CS20]). Let G be a gadget and let P be a set of t_i probes placed on internal wires of G . Let A be a set of t_o output-share indices. The gadget G is said to satisfy d -PINI if, for every configuration of P and A such that $t_i + t_o \leq d$, there exists a set B of at most t_i share indices such that the probes on the wires set $P \cup Y_A$ can be simulated by the input shares $X_{A \cup B}$. Here, $X_{A \cup B}$ denotes all inputs of share indices in set $A \cup B$, and Y_A denotes all outputs of share indices in set A .

Definition 3 (d -Output-Probe-Isolating Non-Interference [CS21]). Let G be a gadget, and let P be a set of t_i probes placed on internal wires of G . Let A be a set of t_o output-share indices. The gadget G is said to satisfy d -OPINI if, for every configuration of P and A such that $t_i + t_o \leq d$, there exists a set B of at most t_i share indices such that the probes on the set wire $P \cup Y_A \cup Y_B$ can be simulated by the input shares $X_{A \cup B}$. Here, $X_{A \cup B}$ denotes all inputs of share indices in set $A \cup B$, Y_A denotes all outputs of share indices in set A , and Y_B denotes all outputs of share indices in set B .

2.4 Related Work

At CHES 2024, Cassiers et al. [CSV24] introduced the first low-latency OPINI gadget, called HPC4. This gadget realized a 2-input multiplication using only a single layer of registers. Owing to its elegant structure, HPC4 can be extended to arbitrary security orders, requiring $\frac{5d(d+1)}{2}$ random bits and $4d^2 + 5d + 1$ registers for security order d . Its first-order instance is shown in Equation (1). Specifically, to achieve OPINI security, a shared term is masked with two random bits inside the HPC4 gadget (e.g., the component function f_2 , where the shared term b_1 is masked with r_0 and r_1 in Equation (1)). These two random bits produces redundant monomials (e.g., a_0r_0 , a_0r_1) during the computation of cross-domain products (e.g., w_1w_2), which should be eliminated by two extra redundant operations (e.g., f_3 and f_4 in Equation (1)).

In contrast to the HPC4 gadget, this work focuses on constructing low-latency, first-order OPINI gadgets applicable to arbitrary Boolean functions. In other words, we extend the capability of OPINI gadgets beyond two-input multiplication to general Boolean functions while preserving a single register layer.

$$\begin{array}{llll}
f_0(a_0, b_0) & = & a_0 b_0 & \rightarrow w_0 \\
f_1(a_0) & = & a_0 & \rightarrow w_1 \\
f_2(b_1, r_0, r_1) & = & b_1 + r_0 + r_1 & \rightarrow w_2 \\
f_3(a_0, r_0, r_2) & = & a_0 r_0 + r_2 & \rightarrow w_3 \\
f_4(a_0, r_1, r_3) & = & a_0 r_1 + r_3 & \rightarrow w_4 \\
\hline
f_5(a_1, b_1) & = & a_1 b_1 & \rightarrow w_5 \\
f_6(a_1) & = & a_1 & \rightarrow w_6 \\
f_7(b_0, r_0, r_4) & = & b_0 + r_0 + r_4 & \rightarrow w_7 \\
f_8(a_1, r_0, r_2) & = & a_1 r_0 + r_2 & \rightarrow w_8 \\
f_9(a_1, r_3, r_4) & = & a_1 r_4 + r_3 & \rightarrow w_9
\end{array}
\quad
\begin{array}{l}
w_1 w_2 + w_0 + w_3 + w_4 = z_0 \\
w_6 w_7 + w_5 + w_8 + w_9 = z_1
\end{array}
\quad (1)$$

Independent and concurrent with our work, Rahimi and Moradi propose OTSM [RM26], a low-latency first-order OPINI gadget, and extend it to arbitrary Boolean functions. In their construction, only the monomials derived from the second input shares are masking using two random bits. In contrast, our construction follows a different design principle by masking all monomials derived from input shares using two random bits and introducing a randomness-reassignment technique that enables reusing randomness across the two output-share computations. Beyond our gadget, we also introduce a symbolic formal verification and apply it to validate multiple low-latency OPINI gadgets, including OTSM.

3 New Techniques

This section presents two new techniques: (i) a method for constructing low-latency first-order OPINI-secure composable gadgets GOM, and (ii) a formal verification technique for validating the security of our OPINI gadgets. Section 3.1 presents a construction method for low-latency OPINI gadgets applicable to arbitrary Boolean functions. In this construction, each shared monomial derived from input shares is masked using two random bits, and a randomness-reassignment technique is employed to further reduce the overall random bits without compromising security. To verify the security of GOM, Section 3.2 presents a symbolic formal verification technique whose key idea is several symbolic reduction rules. Leveraging these rules, the verification technique is applicable to existing low-latency OPINI gadgets, including HPC4, GOM and OTSM gadgets.

3.1 Low-Latency First-Order OPINI Gadgets

To enhance clarity, Section 3.1.1 initially illustrates our technique through an example of a first-order OPINI-secure AND3 gadget. Subsequently, we extend this method to construct low-latency first-order OPINI gadgets for arbitrary Boolean functions, accompanied by security proofs in Section 3.1.2.

3.1.1 A First-Order OPINI Gadget for the AND3 Gate

The first-order OPINI gadget for the AND3 gate (i.e., $z = f(a, b, c) = abc$) is expressed using Boolean expressions, as shown in Equations (2) and (3). Equation (2) specifies the operations before the register layer (i.e., w_0 – w_{15}), while Equation (3) describes the operations after the register layer. Specifically, the Boolean functions f_0 – f_7 generate the first output share z_0 , and are therefore said to operate in domain 0. Similarly, f_8 – f_{15} are performed in domain 1. In domain 0, the shared monomials from domain 1 (i.e., b_1 , c_1 , b_1c_1) are masked using (r_0, r_1) , (r_2, r_3) , and (r_4, r_5) , respectively. To ensure the overall correctness of the first-order OPINI AND3 gadget, two extra redundant operations (i.e., f_6 and f_7) are needed in domain 0. Similarly, in domain 1, the shared monomials b_0 , c_0 ,

b_0c_0 are masked with (r_0, r_8) , (r_2, r_9) , (r_4, r_{10}) , respectively, and the redundant operations f_{14} and f_{15} for the output share z_1 .

$$\begin{array}{llll}
f_0(a_0) & = & a_0 & \rightarrow w_0 \\
f_1(b_0) & = & b_0 & \rightarrow w_1 \\
f_2(c_0) & = & c_0 & \rightarrow w_2 \\
f_3(b_1, r_0, r_1) & = & b_1 + r_0 + r_1 & \rightarrow w_3 \\
f_4(c_1, r_2, r_3) & = & c_1 + r_2 + r_3 & \rightarrow w_4 \\
f_5(b_1, c_1, r_4, r_5) & = & b_1c_1 + r_4 + r_5 & \rightarrow w_5 \\
f_6(a_0, b_0, c_0, r_0, r_2, r_4, r_6) & = & a_0b_0c_0 + a_0c_0r_0 + a_0b_0r_2 + a_0r_4 + r_6 & \rightarrow w_6 \\
f_7(a_0, b_0, c_0, r_1, r_3, r_5, r_7) & = & a_0c_0r_1 + a_0b_0r_3 + a_0r_5 + r_7 & \rightarrow w_7 \\
\hline
f_8(a_1) & = & a_1 & \rightarrow w_8 \\
f_9(b_1) & = & b_1 & \rightarrow w_9 \\
f_{10}(c_1) & = & c_1 & \rightarrow w_{10} \\
f_{11}(b_0, r_0, r_8) & = & b_0 + r_0 + r_8 & \rightarrow w_{11} \\
f_{12}(c_0, r_2, r_9) & = & c_0 + r_2 + r_9 & \rightarrow w_{12} \\
f_{13}(b_0, c_0, r_4, r_{10}) & = & b_0c_0 + r_4 + r_{10} & \rightarrow w_{13} \\
f_{14}(a_1, b_1, c_1, r_0, r_2, r_4, r_6) & = & a_1b_1c_1 + a_1c_1r_0 + a_1b_1r_2 + a_1r_4 + r_6 & \rightarrow w_{14} \\
f_{15}(a_1, b_1, c_1, r_7, r_8, r_9, r_{10}) & = & a_1c_1r_8 + a_1b_1r_9 + a_1r_{10} + r_7 & \rightarrow w_{15}
\end{array} \tag{2}$$

$$\begin{array}{ll}
\frac{w_0w_1w_4 + w_0w_3w_2 + w_0w_5 + w_6 + w_7}{w_8w_9w_{12} + w_8w_{11}w_{10} + w_8w_{13} + w_{14} + w_{15}} & = z_0 \\
& = z_1
\end{array} \tag{3}$$

The above example illustrates the first technique used to construct the OPINI gadget, i.e., masking each shared monomial (i.e., $b_{0/1}, c_{0/1}, b_0c_0, b_1c_1$) with two random bits. To guarantee OPINI security, the above random bits (i.e., $r_0, r_1, r_2, r_3, r_4, r_5, r_8, r_9, r_{10}$) can be organized into two categories. The first category consists of the common random bits r_0, r_2, r_4 , which are shared between the two circuits producing the two output shares; The second category contains distinct random bits (i.e., $r_1, r_3, r_5, r_8, r_9, r_{10}$), which are used independently to mask shared monomials. Noted that the idea of masking each shared monomial using two random bits is an extension of the HPC4 idea. In the HPC4 gadget, each share of one secret input is masked using two random bits (e.g., the share b_1 (resp., b_0) is masked with r_0 and r_1 (resp., r_0 and r_4) in Equation 1). To apply to the AND3 gadget, we extend the masked objective from input shares to shared monomials (e.g., the $b_{0/1}, c_{0/1}, b_0c_0, b_1c_1$). By extending the HPC4 idea, this work aims to construct OPINI-secure gadgets for the AND3 gate and even for arbitrary Boolean functions.

On the premise of achieving OPINI security², we further introduce an optimized gadget that employs a randomness-reassignment technique to reduce the number of random bits. In this optimized construction, the random bits required by the two output-share circuits are reused, resulting in an AND3 gadget that requires only 8 random bits. The optimized scheme is shown in Equations (4) and (3). Compared with Equation (2), the main differences in Equation (4) lie in the operations of f_{11} – f_{13} . Specifically, we modify the ordering of random bits in f_3 – f_5 to mask f_{11} – f_{13} . To elaborate, if the random bits masking f_3 – f_5 are concatenated into a vector $R = r_0\|r_1\|r_2\|r_3\|r_4\|r_5$, then the random bits masking f_{11} – f_{13} are obtained by rotating R left by one position, yielding $R' = r_1\|r_2\|r_3\|r_4\|r_5\|r_0$. Consequently, the shared monomials b_0, c_0 , and b_0c_0 are masked with (r_1, r_2) , (r_3, r_4) , and (r_5, r_0) , respectively. Following this modification, the two redundant functions f_{14} and f_{15} are adjusted accordingly. The operations performed after the register layer remain unchanged and still use Equation (3).

²Since the formal proof is provided in Section 3.1.2, the security proof for this scheme is omitted here.

$$\begin{array}{llll}
f_0(a_0) & = & a_0 & \rightarrow w_0 \\
f_1(b_0) & = & b_0 & \rightarrow w_1 \\
f_2(c_0) & = & c_0 & \rightarrow w_2 \\
f_3(b_1, r_0, r_1) & = & b_1 + r_0 + r_1 & \rightarrow w_3 \\
f_4(c_1, r_2, r_3) & = & c_1 + r_2 + r_3 & \rightarrow w_4 \\
f_5(b_1, c_1, r_4, r_5) & = & b_1c_1 + r_4 + r_5 & \rightarrow w_5 \\
f_6(a_0, b_0, c_0, r_0, r_2, r_4, r_6) & = & a_0b_0c_0 + a_0c_0r_0 + a_0b_0r_2 + a_0r_4 + r_6 & \rightarrow w_6 \\
f_7(a_0, b_0, c_0, r_1, r_3, r_5, r_7) & = & a_0c_0r_1 + a_0b_0r_3 + a_0r_5 + r_7 & \rightarrow w_7 \\
\hline
f_8(a_1) & = & a_1 & \rightarrow w_8 \\
f_9(b_1) & = & b_1 & \rightarrow w_9 \\
f_{10}(c_1) & = & c_1 & \rightarrow w_{10} \\
f_{11}(b_0, r_1, r_2) & = & b_0 + r_1 + r_2 & \rightarrow w_{11} \\
f_{12}(c_0, r_3, r_4) & = & c_0 + r_3 + r_4 & \rightarrow w_{12} \\
f_{13}(b_0, c_0, r_0, r_5) & = & b_0c_0 + r_5 + r_0 & \rightarrow w_{13} \\
f_{14}(a_1, b_1, c_1, r_1, r_3, r_5, r_6) & = & a_1b_1c_1 + a_1c_1r_1 + a_1b_1r_3 + a_1r_5 + r_6 & \rightarrow w_{14} \\
f_{15}(a_1, b_1, c_1, r_0, r_2, r_4, r_7) & = & a_1c_1r_2 + a_1b_1r_4 + a_1r_0 + r_7 & \rightarrow w_{15}
\end{array} \tag{4}$$

Finally, after optimization, the low-latency first-order OPINI AND3 gadget requires only 8 random bits and 16 registers. In the next subsection, we will demonstrate the generality of our techniques by extending them to arbitrary Boolean functions.

3.1.2 The First-Order OPINI Gadget for an Arbitrary Boolean Function

An arbitrary Boolean function can be represented in Algebraic Normal Form (ANF) as an XOR sum of multiple monomials. Since XOR operations can be trivially implemented within the OPINI masking framework, we apply our technique to each monomial to construct a low-latency, first-order OPINI gadget. By integrating these masked monomials, we obtain a complete masking scheme for any Boolean function. We will describe the construction guideline, construction method, security proof, and the method of integrating into an arbitrary Boolean function in the following.

Construction Guidelines. To construct a low-latency, first-order OPINI gadget for an arbitrary monomial, we propose several key guidelines, outlined below.

- **Low-Latency.** This requires the masking scheme to contain only a single layer of registers. In addition, the critical path—defined as the maximum delay between the paths from the primary inputs to the registers and from the registers to the final outputs—should be as short as possible.
- **Low-Randomness.** To minimize randomness usage, we introduce a randomness-reassignment technique that allows circuits producing two output shares to securely reuse the same set of random bits. For example, in the AND3 gadget shown in Equation (4), the random bits $r_0, r_1, \dots, r_7$ is shared between z_0 and z_1 .
- **Correctness.** The XOR sum of all output shares equals the correct secret output. For the AND3 gadget $z = f(a, b, c) = abc$ in Equations (4) and (3), the two output shares are $z_0 = w_0w_1w_4 + w_0w_3w_2 + w_0w_5 + w_6 + w_7 = a_0bc + r_6 + r_7$ and $z_1 = w_8w_9w_{12} + w_8w_{11}w_{10} + w_8w_{13} + w_{14} + w_{15} = a_1bc + r_6 + r_7$. Their XOR sum satisfies $z = z_0 + z_1$, ensuring correctness of the overall computation.
- **Uniformity.** This property requires that each output share be masked with at least one random bit. We ensure this by assigning one random bit to each redundant function of output shares (e.g., f_6 and f_7 , f_{14} and f_{15} in the AND3 gadget of Equation (4)), using the random bits r_6 and r_7 .

- **OPINI security.** According to the definition of first-order OPINI, we consider two types of probing: intermediate probing (i.e., probing the inputs of registers) and output probing (i.e., probing the output shares). When a probe is placed at the input of a register, we can observe certain secret shares from a single domain—denoted as domain B—along with some random bits. Based on this known information, all registers contributing to the output share of domain B should be simulated using only secret shares from the same domain B, and some random bits. To satisfy this requirement, we introduce an additional dedicated random bit to remask each shared monomial, preventing the leakage of cross-domain variables (i.e., some input shares not from domain B). For example, in the AND3 gadget of Equation (4), a probe at the input of register w_6 reveals a_0, b_0, c_0 , and r_0, r_2, r_4, r_6 , i.e., some secret shares from domain 0 and some random bits. The registers w_0 – w_7 generating z_0 can be simulated using the above known information along with the additional random bits r_1, r_3, r_5, r_7 . When a probe is placed at an output share from a certain domain (denoted as domain A), this case should be simulated using certain secret shares from domain A and some random bits. To satisfy this requirement, each shared monomial derived from the input shares should be masked with at least one distinct random bit. For instance, a probe at the output share z_0 reveals a_0, b_0, c_0 , and $r_0/r_1, r_2/r_3, r_4/r_5, r_6, r_7$.

The above guidelines establish the core requirements for the low-latency OPINI gadget. Building upon them, we present the construction method of the first-order OPINI gadget for an arbitrary monomial in the following.

Construction Method. To clearly describe the method for constructing a first-order OPINI gadget for an arbitrary monomial, we present the construction steps in Algorithm 1.

Algorithm 1 takes five inputs: the target monomial $\prod_{i=0}^{m-1} x^i$ with $m > 2$, input shares X_0 and X_1 , a randomness vector R , and 2-bit randomness r' and r'' . The algorithm outputs the desired gadget circuit with output shares z_0 and z_1 . The first two lines define the set Ω as the collection of all non-empty subsets of $\{1, 2, \dots, m-1\}$. The element number of Ω is $\ell = 2^{m-1} - 1$. To reference its elements in the following, we take an enumeration $\{U_t\}$ of Ω and use t as the index, where $0 \leq t \leq \ell - 1$. Lines 3 to 5 specify the usage of randomness. Specifically, the randomness vector R is assigned to $r^{(0,t,1)}$ and $(r^{(0,t,2)})$, which are involved in operations in domain 0, and used to mask the shared monomials derived from the input shares with index 1. The function $\text{ROTATELEFTBYONE}(R)$ denotes a one-position left rotation of R ; the rotated result is assigned to $r^{(1,t,1)}$ and $(r^{(1,t,2)})$, which participate in operations in domain 1, and mask the shared monomials from the input shares with index 1. The random bit r' (resp., r'') is assigned to $r^{(0,1)}$ and $r^{(1,1)}$ (resp., $r^{(0,2)}$ and $r^{(1,2)}$), and is used to mask the redundant functions in domain 0 (resp., domain 1). Lines 6 to 8 synchronize each shared term by storing it into register $\mathcal{S}^{(i,j)}$. In lines 10 to 14, the symbol U_t represents the t -th element of the set Ω , while V_t denotes the complement of U_t . The required shared monomials derived from the input shares with index $1 - i$ are represented as $\prod_{u \in U_t} x_{1-i}^u$, which are masked with two random bits $r^{(i,t,1)}$ and $r^{(i,t,2)}$, respectively. The masked result is then stored into register $\mathcal{P}^{(i,t)}$. The variables $n^{(i,t,1)}$ and $n^{(i,t,2)}$ are auxiliary intermediate variables used to generate the following two redundant registers. Note that the variable $\prod_{v \in V_t} x_i^v$ in the above intermediate variables equals 1 when the set V_t is empty. Lines 16 to 17 generate two redundant registers $\mathcal{N}^{(i,1)}$ and $\mathcal{N}^{(i,2)}$, whose stored values are already masked with $r^{(i,1)}$ and $r^{(i,2)}$, respectively. Lines 19 to 22 generate the required cross-domain product $w^{(i,t)}$. The output share z_i is obtained in line 24 by summing all cross-domain products and the two redundant registers.

The construction guidelines are also annotated in Algorithm 1. Lines 8, 12, 16 and 17 ensure the low-latency guideline, meaning that this scheme contains only a single layer of registers. The low-randomness guideline is achieved in lines 3 to 5, where one set of random bits is reused between circuits producing two output shares. In lines 16, 17 and 24,

two random bits $r^{(i,1)}$ and $r^{(i,2)}$, are involved in the output share z_i , thereby satisfying the uniformity. Lines 8, 12, 16 and 17 jointly guarantee the OPINI security. Line 24 eliminates the redundant monomials introduced during the computation of cross-domain product $w^{(i,t)}$ by performing XOR operations on $\mathcal{N}^{(i,1)}$ and $\mathcal{N}^{(i,2)}$. This ensures correctness, meaning the XOR sum of two output shares, z_0 and z_1 , equals the target monomial.

Algorithm 1 The first-order OPINI gadget for a monomial of degree three or higher

Input: the target monomial $\prod_{i=0}^{m-1} x^i$ with $m > 2$,
input share $X_0 = x_0^{m-1} \| x_0^{m-2} \| \dots \| x_0^0$,
input share $X_1 = x_1^{m-1} \| x_1^{m-2} \| \dots \| x_1^0$,
a randomness vector R of length $2^m - 2$,
2-bit randomness r', r'' .

Output: the circuit with output shares z_0, z_1 , such that $z_0 + z_1 = \prod_{i=0}^{m-1} x^i$.

- 1: $\Omega \leftarrow \{U \subseteq \{1, 2, \dots, m-1\} \mid U \neq \emptyset\}$
- 2: $\ell \leftarrow |\Omega| (= 2^{m-1} - 1)$ $\triangleright \Omega \leftarrow \{U_t\}, 0 \leq t \leq \ell - 1$
- 3: $\langle r^{(0,t,1)}, r^{(0,t,2)} \rangle_{0 \leq t \leq \ell-1} \leftarrow R$
 $\triangleright \langle r^{(0,t,1)}, r^{(0,t,2)} \rangle_{0 \leq t \leq \ell-1}$ cascades all 2-bit blocks $(r^{(0,t,1)}, r^{(0,t,2)})$.
- 4: $\langle r^{(1,t,1)}, r^{(1,t,2)} \rangle_{0 \leq t \leq \ell-1} \leftarrow \text{ROTATELEFTBYONE}(R)$
 $\triangleright \text{ROTATELEFTBYONE}(R)$ denotes a one-bit cyclic left rotation of R .
- 5: $r^{(0,1)} = r^{(1,1)} \leftarrow r'; r^{(0,2)} = r^{(1,2)} \leftarrow r''$ $\triangleright$ Low-randomness
- 6: **for** $i = 0$ to 1 **do**
- 7: **for** $j = 0$ to $m-1$ **do**
- 8: $\mathcal{S}^{(i,j)} \leftarrow \text{Reg}_{\text{pipe}}[x_i^j]$ $\triangleright$ Low-Latency, OPINI security
- 9: **end for**
- 10: **for** $t = 0$ to $\ell - 1$ **do**
- 11: $V_t \leftarrow \{1, 2, \dots, m-1\} \setminus U_t$
- 12: $\mathcal{P}^{(i,t)} \leftarrow \text{Reg}[\prod_{u \in U_t} x_{1-i}^u + r^{(i,t,1)} + r^{(i,t,2)}]$ $\triangleright$ Low-Latency, OPINI security
- 13: $n^{(i,t,1)} \leftarrow x_i^0 \cdot \prod_{v \in V_t} x_i^v \cdot r^{(i,t,1)}$
- 14: $n^{(i,t,2)} \leftarrow x_i^0 \cdot \prod_{v \in V_t} x_i^v \cdot r^{(i,t,2)}$
- 15: **end for**
- 16: $\mathcal{N}^{(i,1)} \leftarrow \text{Reg}[\prod_{j=0}^{m-1} x_i^j + \sum_{t=0}^{\ell-1} n^{(i,t,1)} + r^{(i,1)}]$ $\triangleright$ Low-Latency, Uniformity, OPINI security
- 17: $\mathcal{N}^{(i,2)} \leftarrow \text{Reg}[\sum_{t=0}^{\ell-1} n^{(i,t,2)} + r^{(i,2)}]$
- 18: **end for**
- 19: **for** $i = 0$ to 1 **do**
- 20: **for** $t = 0$ to $\ell - 1$ **do**
- 21: $V_t \leftarrow \{1, 2, \dots, m-1\} \setminus U_t$
- 22: $w^{(i,t)} \leftarrow \mathcal{S}^{(i,0)} \cdot \prod_{v \in V_t} \mathcal{S}^{(i,v)} \cdot \mathcal{P}^{(i,t)}$
- 23: **end for**
- 24: $z_i = \sum_{t=0}^{\ell-1} w^{(i,t)} + \mathcal{N}^{(i,1)} + \mathcal{N}^{(i,2)}$ $\triangleright$ Correctness
- 25: **end for**

As for the performance of an arbitrary masked monomial, the number of required random bits and registers is given below. According to line 12 of Algorithm 1, the number of U_t is $\ell = 2^{m-1} - 1$, which requires $2 \times (2^{m-1} - 1) = 2^m - 2$ random bits. In addition, two random bits r' and r'' are used in lines 3–5 of Algorithm 1. Therefore, for a masked monomial consisting of m terms ($m > 2$), a total of $2^m - 2 + 2 = 2^m$ random bits is required. Since the registers are used not only to store variables masked with the random bits described above, but also to store the shared terms in line 8 of Algorithm 1, the number of registers contributing to one output share is $2^{m-1} - 1 + 2 + m = 2^{m-1} + m + 1$. Therefore, the total number of registers required for the complete masking construction is

$$2 \times (2^{m-1} + m + 1) = 2^m + 2m + 2.$$

Since Algorithm 1 is defined under the constraint $m > 2$, we also provide the $m = 2$ case for completeness, as shown in Equation (5). The main difference from Algorithm 1 is that the length of the randomness vector R is extended to three bits, $R = r_0 \| r_1 \| r_2$, instead of $2^2 - 2 = 2$ bits. The $m = 2$ case requires 5 random bits and 8 registers. Compare with the first-order instance of the HPC4 gadget, our gadget reduces two registers by merging the monomial $a_0 b_0$ (resp., $a_1 b_1$) into the operation f_2 (resp., f_6). This merging yields a 18% reduction in hardware area, as shown in Table 1.

$$\begin{array}{llll}
f_0(a_0) & = & a_0 & \rightarrow w_0 \\
f_1(b_1, r_0, r_1) & = & b_1 + r_0 + r_1 & \rightarrow w_1 \\
f_2(a_0, b_0, r_0, r_3) & = & a_0 b_0 + a_0 r_0 + r_3 & \rightarrow w_2 \\
f_3(a_0, r_1, r_4) & = & a_0 r_1 + r_4 & \rightarrow w_3 \\
\hline
f_4(a_1) & = & a_1 & \rightarrow w_4 \\
f_5(b_0, r_1, r_2) & = & b_0 + r_1 + r_2 & \rightarrow w_5 \\
f_6(a_1, b_1, r_1, r_3) & = & a_1 b_1 + a_1 r_1 + r_3 & \rightarrow w_6 \\
f_7(a_1, r_2, r_4) & = & a_1 r_2 + r_4 & \rightarrow w_7
\end{array}
\quad \begin{array}{l} \\ \\ \\ \\ \\ \\ \\ \end{array}
\quad \begin{array}{l} \\ \\ \\ \\ w_0 w_1 + w_2 + w_3 = z_0 \\ w_4 w_5 + w_6 + w_7 = z_1 \end{array} \quad (5)$$

Security Proof. For any masked monomial generated by Algorithm 1, the OPINI security proof is presented below.

Theorem 1. *Assuming that all randomness bits involved in Algorithm 1 are statistically independent of each input share, then the masked monomial generated by Algorithm 1 is iterated glitch+transition-robust 1-OPINI.*

Proof.

- i. When a probe is placed at the input of $\mathcal{S}^{(i,j)}$, it observes the variable x_i^j , which can be directly simulated by x_i^j . There are four types of registers contributing to the output share z_i , namely $\mathcal{S}^{(i,j)}$, $\mathcal{P}^{(i,t)}$, $\mathcal{N}^{(i,1)}$, and $\mathcal{N}^{(i,2)}$. Since the registers $\mathcal{P}^{(i,t)}$, $\mathcal{N}^{(i,1)}$, and $\mathcal{N}^{(i,2)}$ are all masked with independent random bits, they can be simulated using $r^{(i,t,1)}/r^{(i,t,2)}$, $r^{(i,1)}$, and $r^{(i,2)}$, respectively. Therefore, the input of $\mathcal{S}^{(i,j)}$ and z_i can be jointly simulated with x_i^j , $r^{(i,t,1)}/r^{(i,t,2)}$, $r^{(i,1)}$, and $r^{(i,2)}$, which is consistent with the OPINI security.
- ii. A probe at the input of $\mathcal{P}^{(i,t)}$ directly reveals variables $\{x_{1-i}^u \mid u \in U_t\}$ together with $r^{(i,t,1)}$ and $r^{(i,t,2)}$. We analyze the four types of registers contributing to z_{1-i} , namely $\mathcal{S}^{(1-i,j)}$, $\mathcal{P}^{(1-i,t)}$, $\mathcal{N}^{(1-i,1)}$, and $\mathcal{N}^{(1-i,2)}$. The value stored in $\mathcal{S}^{(1-i,j)}$ can be simulated using x_{1-i}^j . By the randomness-reassignment technique, the two known random bits $r^{(i,t,1)}$ and $r^{(i,t,2)}$ are reused in two different instances of $\mathcal{P}^{(1-i,t)}$, denoted as $\mathcal{P}^{(1-i,t_\alpha)}$ and $\mathcal{P}^{(1-i,t_\beta)}$, such that $r^{(1-i,t_\alpha,2)} = r^{(i,t,1)}$ and $r^{(1-i,t_\beta,1)} = r^{(i,t,2)}$. Since these two instances still contains unknown random bits, namely $r^{(1-i,t_\alpha,1)}$ and $r^{(1-i,t_\beta,2)}$, the values stored in $\mathcal{P}^{(1-i,t_\alpha)}$ and $\mathcal{P}^{(1-i,t_\beta)}$ can be simulated using these unknown random bits, respectively. For all other instances $\mathcal{P}^{(1-i,t)}$ with $t \notin \{t_\alpha, t_\beta\}$, which do not involve $r^{(i,t,1)}$ and $r^{(i,t,2)}$, simulation can be performed using $r^{(1-i,t,1)}/r^{(1-i,t,2)}$. The values stored in $\mathcal{N}^{(1-i,1)}$ and $\mathcal{N}^{(1-i,2)}$ are masked with two unknown random bits $r^{(1-i,1)}$ and $r^{(1-i,2)}$, respectively. They can be simulated using these two bits, respectively. Therefore, this case can be jointly simulated with multiple x_{1-i}^u , $r^{(i,t,1)}$, $r^{(i,t,2)}$, x_{1-i}^j , $r^{(1-i,t_\alpha,1)}$, $r^{(1-i,t_\beta,2)}$, $r^{(1-i,t,1)}/r^{(1-i,t,2)}$ (for $t \notin \{t_\alpha, t_\beta\}$), $r^{(1-i,1)}$, and $r^{(1-i,2)}$, which is consistent with the OPINI security.
- iii. When a probe at the input of $\mathcal{N}^{(i,1)}$ directly reveals multiple x_i^j , $n^{(i,t,1)}$ and $r^{(i,1)}$. Since $n^{(i,t,1)}$ is the product of x_i^0 , $\prod_{v \in V} x_i^v$ and $r^{(i,t,1)}$, it can be simulated using

multiple $i - th$ shares of secret variables x and random bit $r^{(i,t,1)}$. These shares correspond to the output share z_i . Following case i, among the four types of registers contributing to z_i , the value of $\mathcal{S}^{(i,j)}$ can be simulated with x_i^j . Since the value stored in $\mathcal{P}^{i,t}$ is masked with two random bits, where $r^{i,t,2}$ remains unknown, it can be simulated using $r^{i,t,2}$. Since the value stored in $\mathcal{N}^{i,2}$ is masked with $r^{(i,2)}$, this can be simulated using $r^{(i,2)}$. Therefore, this case can be simulated with multiple shares with index i , multiple $r^{(i,t,1)}$, $r^{(i,1)}$, x_i^j , $r^{(i,t,2)}$, and $r^{(i,2)}$.

- iv. A probe at the input of $\mathcal{N}^{(i,2)}$ directly reveals multiple x_i^j , $n^{(i,t,2)}$ and $r^{(i,2)}$. It is similar with case iii. This case can be simulated using multiple shares with index i , multiple $r^{(i,t,2)}$, $r^{(i,2)}$, x_i^j , $r^{(i,t,1)}$, and $r^{(i,1)}$.
- v. When a probe is placed at the output share z_i , it observes several registers, including $\mathcal{S}^{(i,v)}$, $\mathcal{P}^{(i,t)}$, $\mathcal{N}^{(i,1)}$, and $\mathcal{N}^{(i,2)}$. Since $\mathcal{P}^{(i,t)}$, $\mathcal{N}^{(i,1)}$, and $\mathcal{N}^{(i,2)}$ are masked with the random bits $r^{(i,t,1)}$ and $r^{(i,t,2)}$, $r^{(i,1)}$, and $r^{(i,2)}$, respectively, this case can be simulated using multiple x_i^j values observed from $\mathcal{S}^{(i,v)}$ together with these random bits. Therefore, this is in line with the OPINI security.

□

Integration into an Arbitrary Boolean Function. To generate first-order, low-latency OPINI circuits for an arbitrary Boolean function, we describe a three-step integration involving generating all masked monomials appearing in the ANF of the target function, generating the combined redundant operations, and generating the combined output shares. For clarity, we exemplify the integration using the Boolean function $f(a, b, c, d) = ab + ad + abc$, corresponding to Equations (6) and (7).

$$\begin{array}{llll}
f_0(a_0) & = & a_0 & \rightarrow w_0 \\
f_1(b_0) & = & b_0 & \rightarrow w_1 \\
f_2(c_0) & = & c_0 & \rightarrow w_2 \\
f_3(b_1, r_0, r_1) & = & b_1 + r_0 + r_1 & \rightarrow w_3 \\
f_4(c_1, r_2, r_3) & = & c_1 + r_2 + r_3 & \rightarrow w_4 \\
f_5(d_1, r_4, r_5) & = & d_1 + r_4 + r_5 & \rightarrow w_5 \\
f_6(b_1, c_1, r_6, r_7) & = & b_1 c_1 + r_6 + r_7 & \rightarrow w_6 \\
f_7(a_0, b_0, c_0, d_0, & = & a_0 b_0 + a_0 d_0 + a_0 b_0 c_0 + & \rightarrow w_7 \\
r_0, r_2, r_4, r_6, r_8) & = & a_0 r_0 + a_0 r_4 + a_0 b_0 r_2 + a_0 c_0 r_0 + a_0 r_6 + r_8 & \\
f_8(a_0, b_0, c_0, d_0, & = & a_0 r_1 + a_0 r_5 + a_0 b_0 r_3 + a_0 c_0 r_1 + a_0 r_7 + r_9 & \rightarrow w_8 \\
r_1, r_3, r_5, r_7, r_9) & & & \\
\hline
f_9(a_1) & = & a_1 & \rightarrow w_9 \\
f_{10}(b_1) & = & b_1 & \rightarrow w_{10} \\
f_{11}(c_1) & = & c_1 & \rightarrow w_{11} \\
f_{12}(b_0, r_1, r_2) & = & b_0 + r_1 + r_2 & \rightarrow w_{12} \\
f_{13}(c_0, r_3, r_4) & = & c_0 + r_3 + r_4 & \rightarrow w_{13} \\
f_{14}(d_0, r_5, r_6) & = & d_0 + r_5 + r_6 & \rightarrow w_{14} \\
f_{15}(b_0, c_0, r_7, r_8) & = & b_0 c_0 + r_7 + r_8 & \rightarrow w_{15} \\
f_{16}(a_1, b_1, c_1, d_1, & = & a_1 b_1 + a_1 d_1 + a_1 b_1 c_1 + & \rightarrow w_{16} \\
r_1, r_3, r_5, r_7, r_8) & = & a_1 r_1 + a_1 r_5 + a_1 b_1 r_3 + a_1 c_1 r_1 + a_1 r_7 + r_8 & \\
f_{17}(a_1, b_1, c_1, d_1, & = & a_1 r_2 + a_1 r_6 + a_1 b_1 r_4 + a_1 c_1 r_2 + a_1 r_8 + r_9 & \rightarrow w_{17} \\
r_0, r_2, r_4, r_6, r_8, r_9) & & &
\end{array} \tag{6}$$

$$\begin{array}{ll}
w_0 w_3 + w_0 w_5 + w_0 w_1 w_4 + w_0 w_2 w_3 + w_0 w_6 + w_7 + w_8 & = z_0 \\
w_9 w_{12} + w_9 w_{14} + w_9 w_{10} w_{13} + w_9 w_{11} w_{12} + w_9 w_{15} + w_{16} + w_{17} & = z_1
\end{array} \tag{7}$$

The integration proceeds in three steps. (i) For an arbitrary Boolean function, we first construct masked monomials using Algorithm 1 for $m > 2$ (e.g., abc) and Equation (5) for $m = 2$ (e.g., ab, ad). This yields multiple registers $\mathcal{S}^{(i,j)}$ that store the input shares, $\mathcal{P}^{(i,t)}$ that store the shared monomials. These correspond to w_0 – w_6 and w_9 – w_{15} in Equation (6) of the example function. (ii) Since the redundant operations generated in the lines 16–17 of Algorithm 1 and Equation (5) are XORed into the output shares (line 24), and the ANF itself is also an XOR-sum of monomials, they can be merged into two combined redundant functions and masked with distinct random bits. In the example, this corresponds to w_7 – w_8 and w_{16} – w_{17} in Equation (6). (iii) Since these cross-domain products contribute to the final output shares through XOR operations (the terms $w^{(i,t)}$ in line 24 of Algorithm 1), they are merged to generate the final output shares z_0 and z_1 . This corresponds to z_0 and z_1 in Equation (7).

For an n -bit Boolean function of algebraic degree $n-1$, the number of required random bits and registers is analyzed as follows. At step (i) of the integration process, at most $2 \times \left\{ \binom{n-1}{1} + \binom{n-1}{2} + \dots + \binom{n-1}{n-2} \right\} = 2^n - 4$ shared monomials need to be remasked, which requires $2 \times (2^{n-1} - 2) = 2^n - 4$ bits of random. Together with two additional random bits used for masking redundant functions, the total randomness requirement is $2^n - 2$ bits. In addition to all the above cases of randomness, registers are also used to store at most $2n$ shared terms generated in the step (i) of the integration, the total number of registers is $2 \times (2^{n-1} - 2 + 2 + n) = 2^n + 2n$.

This construction can be naturally extended from Boolean functions to vectorial Boolean functions by reusing the registers $\mathcal{S}^{(i,j)}$ and $\mathcal{P}^{(i,t)}$. For an n -dimensional vectorial Boolean function, only an additional $2(n-1)$ -bit randomness is required to mask the redundant functions of the remaining coordinate functions. Consequently, for a $\mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ vectorial Boolean function of algebraic degree $n-1$, our scheme requires at most $2^n + 2n - 4$ bits of random and $2^n + 6n - 4$ registers.

3.2 Formal Verification for Selected Gadgets

To the best of our knowledge, there is no public formal verification tool for OPINI gadgets. To verify the OPINI security of our gadgets, we introduce a symbolic verification technique. Rather than serving as a general formal verification tool for arbitrary circuit, our technique is designed as a lightweight front-end pre-verification module that can efficiently validate OPINI security for a specific type of circuits, which contain a single layer of registers. By leveraging symbolic reasoning, the method operates extremely efficiently and can even handle complex designs, such as the AES S-box.

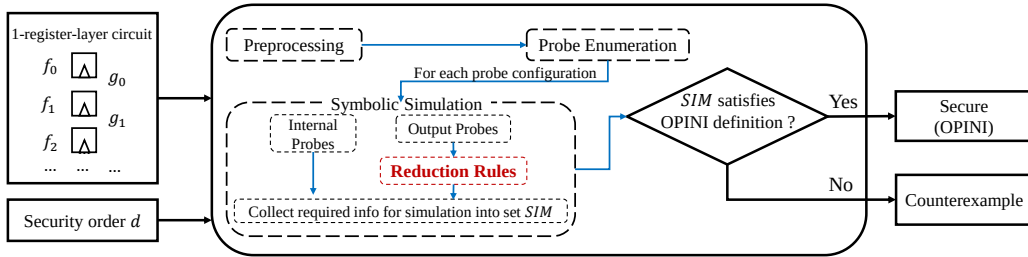


Figure 1: The operation process of our formal verification module.

The workflow of our formal verification module is shown in Figure 1. The input is a 1-register-layer circuit, consisting of a single register layer, a pre-register combinational circuit generating intermediate signals $\mathcal{F} = \{f_0, f_1, \dots\}$, and a post-register combinational circuit producing final outputs $\mathcal{G} = \{g_0, g_1, \dots\}$. This circuit can be parsed from a hardware

netlist. The final output of the formal verification module is the security verdict result or a counterexample. If and only if, for every probe configuration, the corresponding simulation information set SIM satisfies the OPINI definition, the output is an OPINI-secure result. Otherwise, the module outputs a probe configuration for which the OPINI check fails.

Preprocessing. For each first-stage $f_i \in \mathcal{F}$, we extract the set of its input variables, denoted by $vars_f[i]$, and the set $r_var[i]$ of linear random variables that do not appear in high-algebraic-degree (≥ 2) monomials. Similarly, for each output function $g_j \in \mathcal{G}$, we derive a set $vars_g[j]$ that stores which outputs of the first-stage functions contribute to the corresponding output function g_j . Taking the AND3 gadget in Equations (4) and (3) as an example, the first-stage functions are $\mathcal{F} = \{f_0, f_1, \dots, f_{15}\}$ and the second-stage functions are $\mathcal{G} = \{z_0, z_1\}$. For the function f_3 , we obtain $vars_f[3] = \{b_1, r_0, r_1\}$ and $r_var[3] = \{r_0, r_1\}$. For the output function z_0 , we have $vars_g[0] = \{f_0, f_1, \dots, f_7\}$.

Probe Enumeration. In this phase, we enumerate only the most powerful internal probing locations (i.e, the outputs of the first-stage functions $\mathcal{F}$), collected in the set I , and the output-share indices, collected in the set A . The total probing locations satisfy $t_i + t_o \leq d$, where t_i is the number of internal probes, t_o is the number of output-share indices and d is the given security order. For example, in a first-order AND3 gadget, one probe may be placed on any output of the combinational functions f_k for $0 \leq k \leq 15$, or on output-share index 0 or 1, corresponding to the position z_0 or z_1 , respectively.

Symbolic Simulation. For each probe configuration, we initial two set SIM and B to store variables for simulation and their corresponding share indices, respectively. Probes placed on the internal locations of the target circuit can be simulated using the input variables of corresponding first-stage functions in $\mathcal{F}$. These variables are then added to the set SIM , and their corresponding share indices are added into the set B . As an illustration, when a probe is placed on the output of the internal function f_3 , all of its input variables $vars_f[3] = \{b_1, r_0, r_1\}$ are added to SIM and the share index $\{1\}$ is added to B . In addition to the simulation of internal probes, output shares with indices in the set $A \cup B$ should also be simulated. These variables required for this simulation are derived and added to SIM using four symbolic reduction rules. These rules are described in the following phase, called reduction rules.

Security Validation. For each probe configuration, we require that the share indices in B satisfy $|B| \leq t_i$. We then derive, from the simulated information set SIM , a set C of share indices and check that $C \subseteq A \cup B$ and the auxiliary set TBD should be empty. If all these conditions hold for every probe configuration, the circuit satisfies OPINI at order d . Otherwise, we output a counterexample configuration for which the OPINI check fails.

Reduction Rules. The reduction rules are proposed to address the problem: deriving the necessary information to simulate output shares with indices in $A \cup B$. The specific method of simulating output probes via reduction rules is described in Algorithm 2. We first identify a set Q , consisting of output functions g_j that contribute to output shares with indices in $A \cup B$. This problem is converted to simulating probe placed on the corresponding output functions $g_i \in Q$. Such output probes can be simulated using the input variables of the functions g_j . Since the input variables are values stored in registers (i.e, the output values of certain functions f_i), this problem is further converted to deriving the information required to simulate the outputs values of these f_i ³. To deal with these output values, we collect the corresponding functions f_i into an auxiliary set TBD . The task of reduction rules is to derive necessary information to simulate the outputs of all functions in TBD . To clearly illustrate these rules, we provide detail descriptions and examples for each rule, together with Algorithm 2, which demonstrates their application.

³Since the output values of f_i are stored in registers, we can only observe their output values, not their input variables.

Algorithm 2 Simulating Output Probes via Reduction Rules

Input: the target circuit consisting sequentially of combinational functions $\mathcal{F} = \{f_i\}$, a layer of registers, combinational functions $\mathcal{G} = \{g_j\}$,
 some information from the preprocessing phase: $\{r_var[i]\}$, $\{vars_f[i]\}$, $\{vars_g[j]\}$,
 the simulated information set SIM ,
 two share indices set A, B ,

Output: the simulated information set SIM and auxiliary set TBD .

```

1:  $TBD \leftarrow \emptyset$ 
2:  $Q \leftarrow \{g_j \in \mathcal{G} \mid g_j \text{ contributes to an output share with index in } A \cup B\}$ 
3: for each  $q$  in  $Q$  do
4:    $TBD \leftarrow TBD \cup vars\_g[q]$ 
5: end for
6:  $size \leftarrow \{1, 2\}$ 
7: while true do
8:    $updated1 \leftarrow false, updated2 \leftarrow false, updated3 \leftarrow false$ 
9:   while true do
10:     $flag \leftarrow false$ 
11:    for each index  $i$  in  $TBD$  do
12:       $U \leftarrow \bigcup_{j \in TBD, j \neq i} vars\_f[j]$ 
13:      find  $r \in r\_var[i]$  such that  $r \notin U$  and  $r \notin SIM$ 
14:      if such an  $r$  exists then ▷ Reduction Rule 1
15:         $SIM \leftarrow SIM \cup \{r\}, TBD \leftarrow TBD \setminus \{i\}$ 
16:         $updated1 \leftarrow true, flag \leftarrow true$ 
17:      end if
18:    end for
19:    if  $flag = false$  then
20:      break
21:    end if
22:  end while
23:  while true do
24:     $flag \leftarrow false$ 
25:    for each subset  $U \subseteq TBD$  with  $|U| \in size$  do
26:       $f' \leftarrow \bigoplus_{i \in U} f_i$ 
27:       $r\_var\_f' \leftarrow (\text{linear “r”-vars of } f') \setminus (\text{higher-degree vars of } f')$ 
28:      if  $r\_var\_f' \subseteq SIM$  then ▷ Reduction Rule 2
29:         $vars\_a \leftarrow \text{all input variables of } f_i, i \in U$ 
30:         $SIM \leftarrow SIM \cup vars\_a, TBD \leftarrow TBD \setminus U$ 
31:         $updated2 \leftarrow true, flag \leftarrow true$ 
32:      end if
33:    end for
34:    if  $flag = false$  then
35:      break
36:    end if
37:  end while
38:  for each subset  $U \subseteq TBD$  with  $|U| \notin size$  and  $|U| \neq 0$  do
39:     $f' \leftarrow \bigoplus_{i \in U} f_i$ 
40:     $r\_var\_f' \leftarrow (\text{linear “r”-vars of } f') \setminus (\text{higher-degree vars of } f')$ 
41:    if  $r\_var\_f' \subseteq SIM$  then ▷ Reduction Rule 3
42:       $size \leftarrow size \cup \{|U|\}$  ▷ update allowed sizes
43:       $updated3 \leftarrow true$ 
44:      break ▷ restart reduction rules 1 and 2 as in lines 7 to 37
45:    end if
46:  end for
47:  if  $updated1 = false$  and  $updated2 = false$  and  $updated3 = false$  then
48:    break
49:  end if
50: end while
51: return ( $SIM, TBD$ )

```


- Reduction Rule 1 (Perfect Mask). This rule, originally introduced in [ZCF25], plays a particularly elegant role in our symbolic reduction. In our setting, if a certain function f_i from the set TBD contains a linear random variable $r \in r_var[i]$ that does not appear in the random-variable set of any other unresolved functions in TBD , and r is not in the set SIM , then f_i can be simulated directly using this unique random variable r . Consequently, r is added to SIM and the index i is removed from TBD . This rule corresponds to lines 7 to 22 of Algorithm 2. The formal description is given below:

$$\begin{aligned} \exists i \in TBD, \exists r \in r_var[i] : & \left(r \notin \bigcup_{\substack{j \in TBD \\ j \neq i}} vars_f[j] \wedge r \notin SIM \right) \\ \implies & f_i \text{ is simulatable using the unique random bit } r. \end{aligned} \quad (8)$$

Example. Consider the AND3 gadget in Equations (4) and (3). When a probe is placed at the output share z_0 , the unresolved set is $TBD = \{0, 1, \dots, 7\}$. Since the linear random variable r_7 appears only in f_7 and in none of the other functions indexed by TBD , Rule 1 applies and yields $SIM \leftarrow SIM \cup \{r_7\}, TBD \leftarrow TBD \setminus \{7\}$. Repeating this reduction, the linear random variables $r_6, r_4/r_5, r_2/r_3$, and r_0/r_1 are sequentially added to SIM , and the corresponding indices 6, 5, 4, 3 are removed from TBD . After exhausting Rule 1, the information for simulation and unresolved sets are $SIM = \{r_7, r_6, r_4/r_5, r_2/r_3, r_0/r_1\}, TBD = \{0, 1, 2\}$.

- Reduction Rule 2 (Multi-function Simulation). This rule enables jointly simulating multiple unresolved functions in TBD via XOR combination. For any subset $U \subseteq TBD$ whose size is permitted by the set $size$, we consider the XOR-combined function $f' \leftarrow \bigoplus_{i \in U} f_i$. If all linear random variables stored in the set r_var_f' are already in SIM , then every $f_i \in U$ can be simulated using its input variables. Consequently, we add these input variables to SIM and the index $i \in U$ is removed from set TBD . This rule corresponds to lines 23 to 37 of Algorithm 2. The formal characterization is given below:

$$\begin{aligned} \exists U \subseteq TBD, |U| \in size : & (r_var_f' \subseteq SIM) \\ \implies & \forall i \in U, f_i \text{ is simulatable using all input variables of } f_i. \end{aligned} \quad (9)$$

Example. For the AND3 gadget after Reduction Rule 1, when the subset size is $|U| = 1$, we sequentially examine the indices $i \in \{0, 1, 2\}$, that is, all elements of TBD . For each such index, the corresponding XOR-combined function f' contains no linear random variables, i.e., $r_var_f' \leftarrow \emptyset$, so the condition $r_var_f' \subseteq SIM$ hold trivially. Therefore, The functions f_0, f_1 , and f_2 are all simulatable. We add their input variables to SIM and remove the corresponding indices from TBD . At this point, the simulation information set becomes $SIM = \{r_7, r_6, r_4/r_5, r_2/r_3, r_0/r_1, a_0, b_0, c_0\}$ and the unsolved set is $TBD = \emptyset$. The allowed subset size $|U| = 2$ is mainly considered for high-order gadgets, where some random bits are typically reused in an antisymmetric form, for example the states $r''_{ij} = -r''_{ji}$ and $r'''_{ij} = -r'''_{ji}$ in the HPC4 gadget (Algorithm 3 in [CSV24]).

- Reduction Rule 3 (Simulation Echo). Since this rule acts as a feedback mechanism for expanding the allowed subset sizes used in Reduction Rules 2. After Rule 2 saturates under the current size set $size = \{1, 2\}$, the unresolved set TBD may still contain multiple functions. Some of these functions may be jointly simulatable via XOR combination for subset sizes that are not yet permitted. To capture such

cases, for each non-empty subset $U \subseteq TBD$ with $|U| \notin size$, we construct the XOR-combined function f' . If all linear random variables stored in the set r_var_f' are already in SIM , then the subset size $|U|$ is added to $size$. Rules 2 are then rerun using the updated $size$. This simulation echo continues until no new subset size can be activated. Rule 3 corresponds to lines 38–50 of Algorithm 2. Its formal characterization is given below:

$$\begin{aligned} \exists U \subseteq TBD, |U| \notin size : r_var_f' \subseteq SIM \\ \implies size \leftarrow size \cup \{|U|\}. \end{aligned} \tag{10}$$

Example. Since all indices in TBD have already been resolved by the previous rules in the AND3 case, the flag variable *update* is *false* during Rule 3, and the rule terminates immediately. Note that Rule 3 serves as an extension mechanism. When the subset size $|U|$ required in Rule 2 is neither 1 nor 2, Rule 3 can activate additional subset sizes and mainly relevant for more complex, higher-order masked circuits.

Leveraging three symbolic reduction rules, the proposed verification module can efficiently handle complex circuits like the AES S-box. This addresses a key shortcoming of general-purpose formal verification tools such as SILVER [KSM20] and VERICA [RFSG22], which often require huge computational and memory costs on large circuits. Though efficient, our verification module is narrower in scope than general-purpose tools and applies only to single-cycle OPINI gadgets (e.g., the GOM and HPC4 gadgets).

4 Case Studies

In this section, we construct the first-order OPINI masking schemes for the S-boxes from the Ascon [DEMS21], PRESENT [BKL⁺07], PRINCE [BCG⁺12], and AES [Dae99] ciphers, and report their hardware performance in Table 1. We then verify the OPINI security of these masked S-boxes using our symbolic verification technique. The corresponding verification results are summarized in Table 2.

4.1 S-box Gadgets

As a standard for lightweight cryptography, Ascon [DEMS21] is designed for efficient and low-cost implementations and is friendly to masking. Its S-box is a vectorial Boolean function over $\mathbb{F}_2^5 \rightarrow \mathbb{F}_2^5$ with algebraic degree 2. Owing to this low degree, the S-box can be implemented using a single layer of AND gates. Based on the five coordinate functions in Equation (11), we apply the technique described in Section 3.1 to construct a single-cycle OPINI gadget for the Ascon S-box. The requirements for random bits and registers are summarized below. Since each shared monomial is masked using two random bits, we require 8 random bits to mask the shared variables b_1 , c_1 , d_1 , and e_1 , and an additional 10 random bits to mask the redundant functions in the circuit contributing to the first output share. Owing to the randomness-reassignment technique, the circuit that produces the second output share can be securely implemented using the same set of 18 random bits. In addition to the registers used to store the values masked by the above random bits, we require 8 additional registers to store the shared inputs $a_{0/1}$, $b_{0/1}$, $c_{0/1}$, and $d_{0/1}$. Consequently, the masked Ascon S-box requires only 18 random bits and 36 registers to achieve OPINI security. Since our technique applies to arbitrary vectorial Boolean functions, it can be trivially extended to S-boxes of different sizes. As representative examples, we also construct masking schemes for the 4-bit S-box (e.g., from PRESENT and PRINCE) and the 8-bit S-box from AES, following processes similar to those used for the Ascon S-box.

$$\begin{aligned}
f(a, b, c, d, e) &= ad + de + a + b + d, \\
f(a, b, c, d, e) &= ae + be + a + b + c + d + e, \\
f(a, b, c, d, e) &= ab + a + c + d + 1, \\
f(a, b, c, d, e) &= bc + bd + cd + a + b + c + d + e, \\
f(a, b, c, d, e) &= ad + cd + de + b + c + d + e.
\end{aligned} \tag{11}$$

To compare our schemes with the state of the art, we synthesize our implementations using Synopsys Design Compiler and the NanGate 45 nm library with the synthesis options: `compile_ultra -no_autoungroup -no_boundary_optimization`. Regarding experimental performance of related work, we use the reported experimental results of LMDPL and CCHPC_{RS} from [SBHM20, LMDM25]. The experimental results for HPC3⁺, HPC4, TSM, and OTSM are obtained by synthesizing the corresponding source codes. HPC4 mainly focuses on the scalability of the 2-input multiplication gadget across arbitrary security orders. The implementation of the first-order masked Ascon S-box provided in the artifact of [CSV24] employs 5 HPC4 gadgets to realize the non-linear functionality and 12 additional registers to synchronize internal signals. Our work concentrates on the construction of the first-order OPINI gadgets for arbitrary Boolean functions. In comparison, our gadget achieves a 21% reduction in hardware area and a 28% reduction in randomness compared to the first-order masked Ascon S-box using HPC4 gadget.

Table 1: Hardware performance of different first-order masked implementations.

Scheme	Security notion	Latency [cycle]	Area [GE]	Delay [ns]	Rand [bit]	Time ^a [ns]	Ref.
AND2 Gadget							
HPC3 ⁺	OPINI	2	95	0.15	3	0.45	[KM22]
HPC4	OPINI	1	94	0.18	5	0.36	[CSV24]
GOM	OPINI	1	77	0.19	5	0.38	This Work
Ascon S-box							
HPC3 ⁺	OPINI	2	661	0.24	15	0.72	[KM22]
HPC4	OPINI	1	583	0.36	25	0.72	[CSV24]
GOM	OPINI	1	460	0.36	18	0.72	This Work
PRESENT S-box							
TSM	PINI	1	427	0.73	17	1.46	[VDB ⁺ 24]
GOM	OPINI	1	569	0.59	20	1.18	This Work
PRINCE S-box							
TSM	PINI	1	450	0.79	18	1.58	[VDB ⁺ 24]
OTSM ^b	OPINI	1	573	0.65	28	1.30	[RM26]
GOM	OPINI	1	631	0.52	20	1.04	This Work
AES S-box							
LMDPL	PINI ^c	1	3480	2.50	36	5.00	[SBHM20]
TSM	PINI	1	14293	1.86	262	3.72	[VDB ⁺ 24]
CCHPC _{RS}	PINI ^c	1	4062	2.22	34	4.44	[LMDM25]
OTSM ^b	OPINI	1	20055	1.89	508	3.78	[RM26]
GOM	OPINI	1	33762	1.89	268	3.78	This Work

^a Time = Delay × (Latency + 1).

^b Independent and concurrent work.

^c Under the robust but relaxed probing model [MM24].

We also report the hardware costs of masked PRESENT S-box [BKL⁺07], PRINCE

S-box [BCG⁺12] and AES S-box [Dae99] in Table 1. For a broader perspective, we also provide several generic low-latency PINI gadgets in Table 1. Note that OPINI offers higher security strength than PINI, as PINI does not account for transition default. Satisfying PINI may not be sufficient to guarantee security in a single-cycle iterative implementation, whereas OPINI is explicitly designed to address this issue. In terms of delay, as shown in Table 1, our GOM gadget outperforms TSM for the PRESENT and PRINCE S-boxes. For AES S-box, our gadget is superior to LMDPL and CCHPC_{RS} in terms of delay, but slightly inferior to TSM.

Compared with the concurrent work OTSM, our gadget reduces the randomness cost by 29% (resp., 47%) and the delay by 20% (resp., 0%) for the PRINCE S-box (resp., AES S-box). These improvements mainly stem from the randomness-reassignment technique. Since our design primarily targets the randomness rather than the hardware area, the area of GOM is inferior to that of OTSM. To assess the total hardware area including randomness generation, we adopt the evaluation methodology of OTSM and instantiate an unrolled Trivium PRNG [CMM⁺24]. For the hybrid PRINCE S-box, the total area (gadget + PRNG) is 3560 GE for OTSM and 3428 GE for GOM. For the hybrid AES S-box, the total area is 56658 GE for OTSM and 54915 GE for GOM. Overall, our hybrid design slightly yields a 3% reduction in total area when accounting for randomness generation.

4.2 Formal Verification of Gadgets

To verify the OPINI security of our masked S-boxes constructed in Section 4.1, we apply the symbolic verification technique proposed in Section 3.2. We begin by describing some implementation details of our verification technique and then provide the verification results for several low-latency OPINI gadgets, including GOM, HPC4 and OTSM.

To implement our verification technique, we develop a symbolic analysis procedure in SageMath. The masked circuits are converted into Algebraic Normal Form (ANF) using SageMath’s `BooleanPolynomialRing`, which supports algebraic degree computation, variable extraction, and polynomial combination. We model probes symbolically by parsing the corresponding ANF expressions, and then apply the reduction rules to derive the information needed to simulate these probes. To improve runtime, we parallelize both the preprocessing and symbolic simulation phases using SageMath’s `multiprocessing` module.

For the masked Ascon S-box, the inputs provided to our verification procedure are the target circuit and the security order $d = 1$. The circuit is characterized by the function set $\mathcal{F} = \{f_0, f_1, \dots, f_{35}\}$, corresponding to the 36 registers discussed in Section 4.1, and the output function set $\mathcal{G} = \{g_0, g_1, \dots, g_9\}$, corresponding to the 10 output wires. Based on the above two inputs, the total number of probe configurations to be verified is $\binom{36}{1} + \binom{10}{1} = 46$. All probe cases are analyzed in parallel using the reduction rules outlined in Algorithm 2. The verification experiments are conducted on a virtual machine equipped with 6 logical cores and 8 GB of RAM, running on an Intel Core i7-8700 CPU at 3.20 GHz. Using the same analysis procedure, we also verify other low-latency OPINI schemes, including our masked PRESENT, PRINCE, and AES S-boxes, as well as the existing HPC4 gadget. In Table 2, we denote the first- and second-order HPC4 gadgets by HPC4 o1 and HPC4 o2, respectively, and the masked Ascon implementation using the first-order HPC4 gadget by HPC4_{Ascon}. As shown in Table 2, the verification time increases with the circuit size and the security order. For the Ascon S-box with 46 probe configurations, the verification completes in approximately 4.05 seconds and confirms the expected OPINI security. However, the masked AES S-box yields $\binom{300}{1} + \binom{16}{1} = 316$ available probe configurations, and the verification requires approximately 19.39 minutes. Moreover, our analysis procedure also verifies the OPINI security of the OTSM-instantiated PRINCE S-box. Due to the large scale of the OTSM-instantiated AES S-box, our procedure does not terminate within 48 h, and thus neither confirms OPINI security nor returns

a counterexample. As a weaker evaluation, we verified the security of eight AES S-box subcircuits (one per output bit), requiring 6.24, 6.70, 6.83, 6.11, 5.54, 4.97, 5.41, and 6.38 minutes, respectively.

Table 2: Verification time of single-cycle OPINI gadgets using our technique

Scheme	Time (This Work)	Scheme	Time (This Work)
GOM _{Ascon} S-box	✓[4.05 s]	HPC4 _{Ascon} S-box	✓[50.38 s]
GOM _{PRESENT} S-box	✓[4.69 s]	HPC4 o1	✓[0.54 s]
GOM _{PRINCE} S-box	✓[5.27 s]	HPC4 o2	✓[1.78 m]
OTSM _{PRINCE} S-box	✓[9.08 s]	GOM _{AES} S-box	✓[19.39 m]

5 Experimental Verification

Although the OPINI security of our gadgets is guaranteed by the security proofs and formal verification techniques, we further conduct FPGA-based experiments to demonstrate the practical security. Specifically, we first develop a general single-cycle iterative architecture and then use it to evaluate our gadgets on an FPGA board.

Since this work focuses on nonlinear components such as S-boxes, we develop a simplified architecture (shown in Figure 2) to evaluate our gadget, similar to the architecture in Figure 13 of [CSV24]. This design simplifies the round functions of block ciphers by retaining only a single S-box from the nonlinear layer and removing the entire linear layer. Because of only a single register layer located inside the S-box gadget, Figure 2 is a single-cycle iterative design consistent with the application scenario of OPINI. FPGA-based experiments are then performed by replacing the S-box instance within this architecture.

The practical experiment is conducted on a SAKURA-G board equipped with a Spartan-6 FPGA as the host core. The clock frequency of this FPGA board is set to 3 MHz. Power traces are captured using a PicoScope 5244D oscilloscope configured with a sampling rate of 250 MS/s. The random bits required by our gadget are generated using an unrolled Trivium [CMM⁺24] implementation. To evaluate the security of our designs, we apply the fixed-versus-random t-test [GGJR⁺11], a widely recognized methodology for assessing the security of masked implementations.

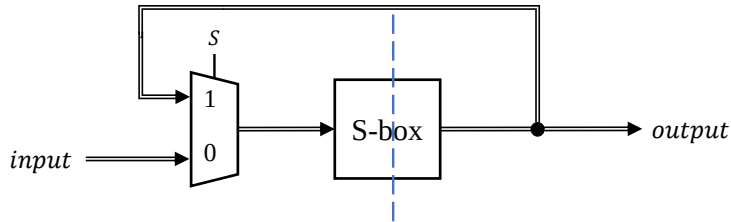


Figure 2: Schematic of the iterative S-box.

To evaluate the masked Ascon S-box, we instantiate it within the architecture shown in Figure 2. Since this gadget is first-order secure, the t-test result using 100 million traces, shown in Figures 3(a), confirms the claimed security level. Figure 3(b) plots the evolution of the maximum absolute t-statistic as the number of traces increases. The first-order statistic consistently stays below the 4.5 threshold across all trace counts, confirming the first-order security. When the PRNGs are disabled, the results using 10 million traces, shown in Figures 3(c) and 3(d), validate our setup by clearly exhibiting first-order leakage.

Similar experiments for the masked PRESENT S-box and PRINCE S-box are also conducted. Their corresponding t-test results, shown in Figures 4 and 5, indicate the claimed first-order security. Due to the unaffordable resource requirement, the masked

AES S-box could not be successfully implemented on the target FPGA board. Nevertheless, given the previously established security proofs, the formal verification results, and the practical experimental results for the masked S-boxes from the Ascon, PRESENT, and PRINCE ciphers, we believe that experiments for the masked AES S-box would yield similar t-test results.

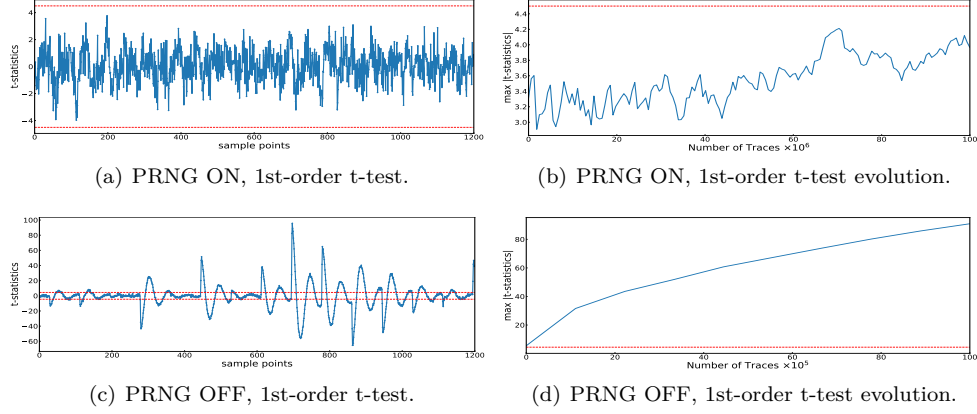


Figure 3: The t-test results for our single-cycle iterative Ascon S-box.

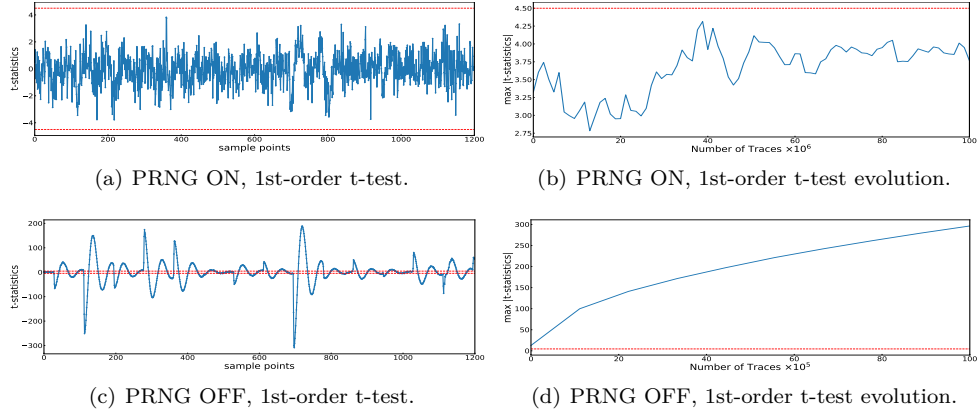


Figure 4: The t-test results for our single-cycle iterative PRESENT S-box.

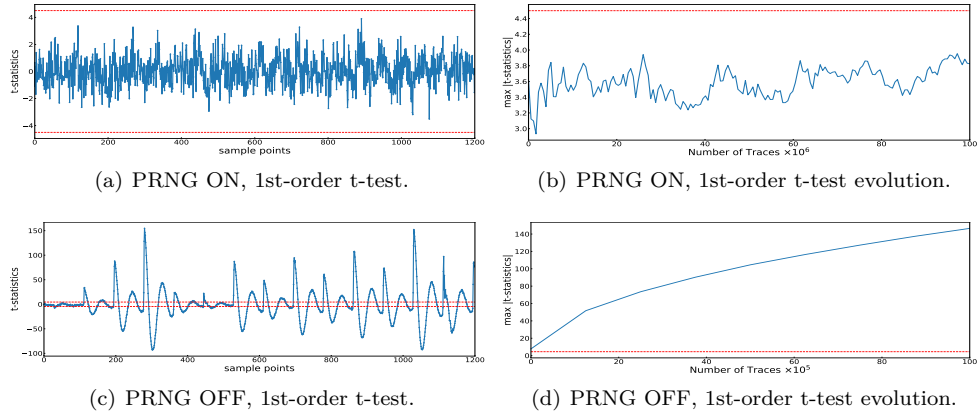


Figure 5: The t-test results for our single-cycle iterative PRINCE S-box.

6 Conclusions

In this work, we present a low-latency first-order OPINI gadget applicable to arbitrary Boolean functions. Its construction is based on two new techniques: extending the HPC4 idea to masking each shared monomial with two random bits and introducing a randomness-reassignment technique. Following the definition of OPINI, this gadget is suitable for single-cycle iterative architectures. Compared with the existing masked Ascon S-box, our gadget achieves reductions of 21% in hardware area and 28% in random bits. To demonstrate the generality of our gadget, we instantiate several high-algebraic-degree (> 2) S-boxes, such as the 3-degree PRESENT S-box and the 7-degree AES S-box, and report their hardware performance as reference baselines. We also propose a symbolic verification technique to confirm the OPINI security of our masked S-boxes within 20 minutes. Finally, we validate the practical security through FPGA-based experiments.

In the future, two aspects can be explored. As shown in Table 1, the security enhancement from robust PINI to robust OPINI leads to a 20%~50% area increase. Further optimizations are needed to reduce the hardware area. Moreover, constructing generic low-latency, high-order OPINI gadgets remains an interesting and challenging direction.

References

- [BBD⁺16] Gilles Barthe, Sonia Belaïd, François Dupressoir, Pierre-Alain Fouque, Benjamin Grégoire, Pierre-Yves Strub, and Rébecca Zucchini. Strong non-interference and type-directed higher-order masking. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, pages 116–129. ACM, 2016.
- [BCG⁺12] Julia Borghoff, Anne Canteaut, Tim Güneysu, Elif Bilge Kavun, Miroslav Knezevic, Lars R. Knudsen, Gregor Leander, Ventzislav Nikov, Christof Paar, Christian Rechberger, Peter Rombouts, Søren S. Thomsen, and Tolga Yalçın. PRINCE - A low-latency block cipher for pervasive computing applications - extended abstract. In Xiaoyun Wang and Kazue Sako, editors, *Advances in Cryptology - ASIACRYPT 2012 - 18th International Conference on the Theory and Application of Cryptology and Information Security, Beijing, China, December 2-6, 2012. Proceedings*, volume 7658 of *Lecture Notes in Computer Science*, pages 208–225. Springer, 2012.
- [BKL⁺07] Andrey Bogdanov, Lars R. Knudsen, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, Yannick Seurin, and C. Vikkelsøe. PRESENT: an ultra-lightweight block cipher. In Pascal Paillier and Ingrid Verbauwhede, editors, *Cryptographic Hardware and Embedded Systems - CHES 2007, 9th International Workshop, Vienna, Austria, September 10-13, 2007, Proceedings*, volume 4727 of *Lecture Notes in Computer Science*, pages 450–466. Springer, 2007.
- [CGLS21] Gaëtan Cassiers, Benjamin Grégoire, Itamar Levi, and François-Xavier Standaert. Hardware private circuits: From trivial composition to full verification. *IEEE Trans. Computers*, 70(10):1677–1690, 2021.
- [CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 398–412. Springer, 1999.
- [CMM⁺24] Gaëtan Cassiers, Loïc Masure, Charles Momin, Thorben Moos, Amir Moradi, and François-Xavier Standaert. Randomness generation for secure hardware masking - unrolled trivium to the rescue. *IACR Commun. Cryptol.*, 1(2):4, 2024.
- [CS20] Gaëtan Cassiers and François-Xavier Standaert. Trivially and efficiently composing masked gadgets with probe isolating non-interference. *IEEE Trans. Inf. Forensics Secur.*, 15:2542–2555, 2020.
- [CS21] Gaëtan Cassiers and François-Xavier Standaert. Provably secure hardware masking in the transition- and glitch-robust probing model: Better safe than sorry. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(2):136–158, 2021.
- [CSV24] Gaëtan Cassiers, François-Xavier Standaert, and Corentin Verhamme. Low-latency masked gadgets robust against physical defaults with application to ascon. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(3):603–633, 2024.

- [Dae99] Joan Daemen. Aes proposal: Rijndael, 1999.
- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *J. Cryptol.*, 34(3):33, 2021.
- [FGP⁺18] Sebastian Faust, Vincent Grosso, Santos Merino Del Pozo, Clara Paglialonga, and Fran  ois-Xavier Standaert. Composable masking schemes in the presence of physical defaults & the robust probing model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):89–120, 2018.
- [GGJR⁺11] Benjamin Jun Gilbert Goodwill, Josh Jaffe, Pankaj Rohatgi, et al. A testing methodology for side-channel resistance validation. In *NIST non-invasive attack testing workshop*, volume 7, pages 115–136, 2011.
- [GMK16] Hannes Gro  , Stefan Mangard, and Thomas Korak. Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order. In Beg  l Bilgin, Svetla Nikova, and Vincent Rijmen, editors, *Proceedings of the ACM Workshop on Theory of Implementation Security, TIS@CCS 2016 Vienna, Austria, October, 2016*, page 3. ACM, 2016.
- [GP99] Louis Goubin and Jacques Patarin. DES and differential power analysis (the "duplication" method). In   etin Kaya Ko   and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems, First International Workshop, CHES'99, Worcester, MA, USA, August 12-13, 1999, Proceedings*, volume 1717 of *Lecture Notes in Computer Science*, pages 158–172. Springer, 1999.
- [KJJ99] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999.
- [KM22] David Knichel and Amir Moradi. Low-latency hardware private circuits. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 1799–1812. ACM, 2022.
- [Koc96] Paul C. Kocher. Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In Neal Koblitz, editor, *Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.
- [KSM20] David Knichel, Pascal Sasdrich, and Amir Moradi. SILVER - statistical independence and leakage verification. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 787–816. Springer, 2020.
- [KSM22] David Knichel, Pascal Sasdrich, and Amir Moradi. Generic hardware private circuits towards automated generation of composable secure gadgets. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(1):323–344, 2022.

- [LMDM25] Daniel Lammers, Nicolai Müller, Siemen Dhooghe, and Amir Moradi. Constant-Cycle Hardware Private Circuits. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(4):172–214, 2025.
- [MCS22] Charles Momin, Gaëtan Cassiers, and François-Xavier Standaert. Handcrafting: Improving automated masking in hardware with manual optimizations. In Josep Balasch and Colin O’Flynn, editors, *Constructive Side-Channel Analysis and Secure Design - 13th International Workshop, COSADE 2022, Leuven, Belgium, April 11-12, 2022, Proceedings*, volume 13211 of *Lecture Notes in Computer Science*, pages 257–275. Springer, 2022.
- [MM24] Nicolai Müller and Amir Moradi. Robust but relaxed probing model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(4):451–482, 2024.
- [NRR06] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold implementations against side-channel attacks and glitches. In Peng Ning, Sihan Qing, and Ninghui Li, editors, *Information and Communications Security, 8th International Conference, ICICS 2006, Raleigh, NC, USA, December 4-7, 2006, Proceedings*, volume 4307 of *Lecture Notes in Computer Science*, pages 529–545. Springer, 2006.
- [RBN⁺15] Oscar Reparaz, Begül Bilgin, Svetla Nikova, Benedikt Gierlichs, and Ingrid Verbauwhede. Consolidating masking schemes. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*, volume 9215 of *Lecture Notes in Computer Science*, pages 764–783. Springer, 2015.
- [RFSG22] Jan Richter-Brockmann, Jakob Feldtkeller, Pascal Sasdrich, and Tim Güneysu. VERICA - verification of combined attacks automated formal verification of security against simultaneous information leakage and tampering. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):255–284, 2022.
- [RM26] Hemin Rahimi and Amir Moradi. TSM+ and OTSM - correct application of time sharing masking in round-based designs. Cryptology ePrint Archive, Paper 2026/004, 2026.
- [SBHM20] Pascal Sasdrich, Begül Bilgin, Michael Hutter, and Mark E. Marson. Low-latency hardware masking with application to AES. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(2):300–326, 2020.
- [Sha79] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, 1979.
- [VDB⁺24] Dilip Kumar S. V., Siemen Dhooghe, Josep Balasch, Benedikt Gierlichs, and Ingrid Verbauwhede. Time sharing - A novel approach to low-latency masking. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024(3):249–272, 2024.
- [VDB⁺25] Dilip Kumar S. V., Siemen Dhooghe, Josep Balasch, Benedikt Gierlichs, and Ingrid Verbauwhede. Higher-order time sharing masking. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(2):235–267, 2025.
- [ZCF25] Feng Zhou, Hua Chen, and Limin Fan. Prover - toward more efficient formal verification of masking in probing model. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2025(1):552–585, 2025.